LLM PERFORMANCE EVALUATION · GROUP 5

ROUGE Score

How much of the human reference does an LLM's summary recover?

HANG Muykhorng
CHOU Kosol
NY Chantharith



ROADMAP

Six parts, one metric

01 **Introduction**
what & why

02 **Theory**
formulas · 6 variants

03 **Practical examples**
two worked cases

04 **Python**
scratch + library

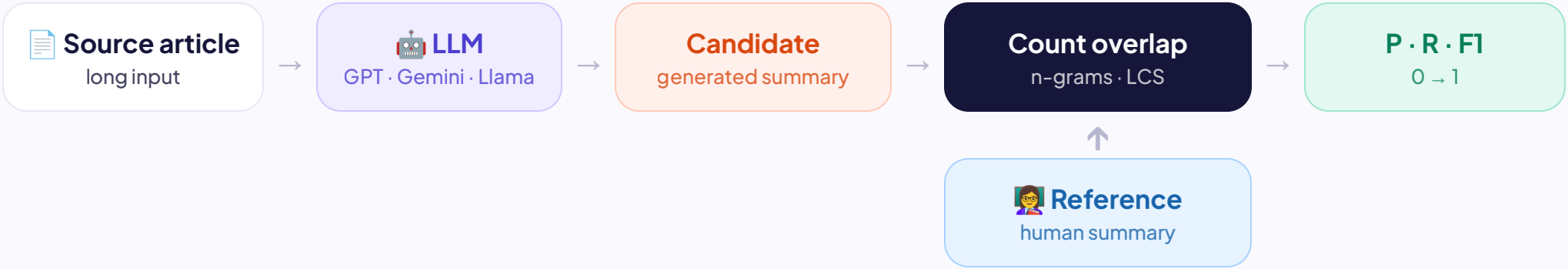
05 **Pros & limits**
strengths · weaknesses

06 **Conclusion**
when · why · where

01 · INTRODUCTION

What is ROUGE, and why do LLMs need it?

Recall-**O**riented **U**nderstudy for **G**isting **E**valuation (Chin-Yew Lin, 2004) compares a machine-generated text with one or more **human reference** texts and counts the **shared words and word sequences**. Scores range from **0** (no overlap) to **1** (perfect overlap).



Scale

Scores thousands of outputs in seconds, where human reading is slow and costly.

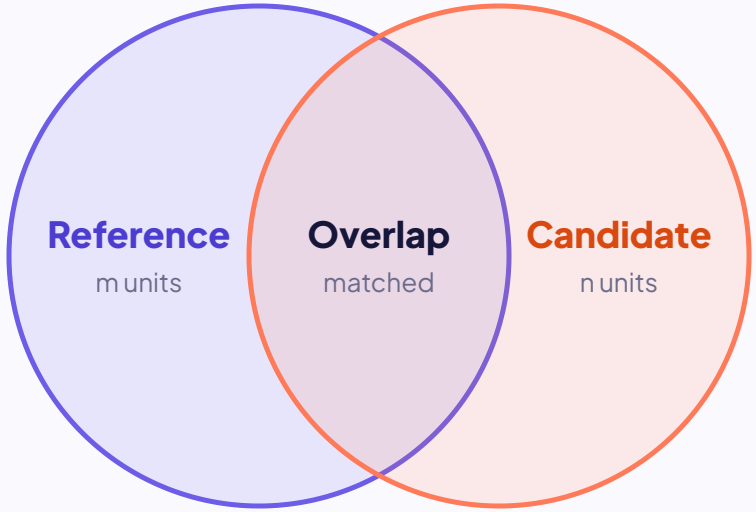
Repeatable

Same inputs give the same score, so model versions can be compared fairly.

Standard

The default metric in summarization research and leaderboards.

The core idea: recall, precision, F1



A “unit” is a word, an n-gram, or a subsequence, depending on the variant.

Recall
How much of the reference did we cover?

$$R = \frac{\text{overlap}}{m}$$

Precision
How much of our output is relevant?

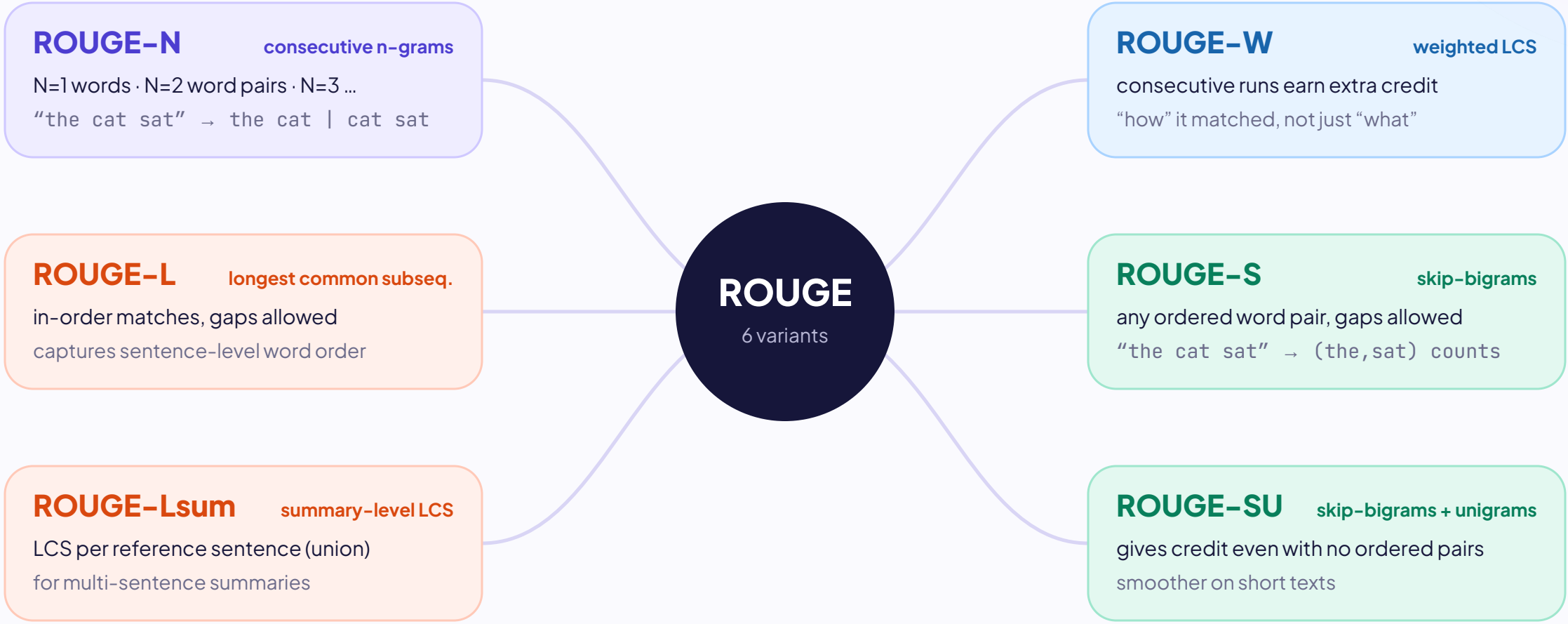
$$P = \frac{\text{overlap}}{n}$$

F1-score
Harmonic mean: balances both

$$F1 = \frac{2 \cdot P \cdot R}{P + R}$$

Original ROUGE was **recall-oriented**. Modern papers usually report **F1** to avoid rewarding long outputs.

The ROUGE family at a glance



Most reported in papers: ROUGE-1, ROUGE-2, ROUGE-L / Lsum

02 · THEORY

ROUGE-N: counting n-gram matches

$$\text{ROUGE-N}_{\text{recall}} = \frac{\sum_{S \in \text{refs}} \sum_{\text{gram}_n \in S} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{S \in \text{refs}} \sum_{\text{gram}_n \in S} \text{Count}(\text{gram}_n)}$$

$\text{Count}_{\text{match}}$

How many times an n-gram appears in **both** texts, **clipped** to the smaller count. “the” twice in the reference and three times in the output counts as 2.

Precision

Same numerator, but divide by the n-grams in the **candidate**. Combine both into F1.

1 **Normalize** lowercase, strip punctuation → 2 **Tokenize** (optional stemming) → 3 **Build n-grams** both texts → 4 **Clip & count** overlap → 5 **P, R, F1** report F1

ROUGE-L & Lsum: longest common subsequence

LCS DYNAMIC-PROGRAMMING TABLE

	the	cat	lay	on	the	mat
the	**1**	1	1	1	1	1
cat	1	**2**	2	2	2	2
sat	1	2	2	2	2	2
on	1	2	2	**3**	3	3
the	1	2	2	3	**4**	4
mat	1	2	2	3	4	**5**

Rows = reference, columns = candidate. The highlighted path gives **the · cat · on · the · mat**, so **LCS = 5**.

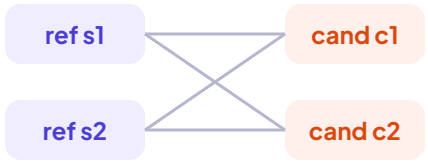
$$R_{lcs} = \frac{LCS(X,Y)}{m} \qquad P_{lcs} = \frac{LCS(X,Y)}{n} \qquad F_{lcs} = \frac{(1+\beta^2)\, R\, P}{R + \beta^2\, P}$$

m = reference length, n = candidate length · β = 1 gives the usual F1

Worked: R = 5/6, P = 5/6 → **ROUGE-L F1 = 0.833**

No fixed N needed, and in-order matches are rewarded automatically.

ROUGE-Lsum (summary-level)



Split into sentences. For **each reference sentence**, take the **union** of its LCS matches with every candidate sentence, then sum. Order *between* sentences no longer matters.

ROUGE-W, ROUGE-S and ROUGE-SU

ROUGE-W · weighted LCS

A run of *k* consecutive matches scores $f(k) = k^2$ instead of k, then $R_W = f^{-1}(WLCS / f(m))$.

```
ref   A B C D E F G
c1    A B C D H I K
c2    A H B K C I D
```

	LCS	R_L	WLCS	R_W
c1	4	0.571	$4^2 = 16$	**0.571**
c2	4	0.571	$4 \times 1^2 = 4$	**0.286**

ROUGE-L can't tell c1 from c2. ROUGE-W rewards the unbroken run.

ROUGE-S · skip-bigrams

Every ordered word pair, with any gap. A text of m words has C(m,2) pairs.

```
ref "the cat sat"
→ the,cat  the,sat  cat,sat
cand "the cat slept"
→ the,cat  the,slept  cat,slept
```

1 match / 3 pairs → R = P = F1 = **0.333**

ROUGE-SU · + unigrams

Adds single words, so reversed text still gets some credit.

```
ref  "A B C"
cand "C B A"
```

UNIT	MATCH	TOTAL
unigrams	3	3
skip-bigrams	0	3

3 / 6 → R = P = F1 = **0.50**

03 · PRACTICAL EXAMPLE 1

One word changed: ROUGE-1 vs ROUGE-2 vs ROUGE-L

REFERENCE

the cat sat on the mat

CANDIDATE

the cat lay on the mat

BIGRAMS (REF → CAND)

the cat | cat sat | sat on | on the | the mat

the cat | cat lay | lay on | on the | the mat

ROUGE-1

5 of 6 words match

0.833

P = R = 5/6

ROUGE-2

3 of 5 pairs match

0.600

P = R = 3/5

ROUGE-L

LCS = 5

0.833

P = R = 5/6

💡 One wrong word breaks **one unigram** but **two bigrams**. That makes ROUGE-2 the strictest score here, and it rewards fluent phrasing.

03 · PRACTICAL EXAMPLE 2

Short vs long output: why we report F1

REFERENCE: `the` `weather` `is` `cold` `outside`

Candidate A · too short

`the` `weather` `is` `cold`

Precision	1.00
Recall	0.80
F1	0.89

ROUGE-1 shown. ROUGE-2: P 1.00 · R 0.75 · F1 0.86

Candidate B · too long

`the` `weather` `is` `cold` `outside` `today` `and` `it` `might` `rain`

Precision	0.50
Recall	1.00
F1	0.67

ROUGE-1 shown. ROUGE-2: P 0.44 · R 1.00 · F1 0.62

💡 Judged on **recall alone**, the padded answer B "wins" with 1.00. **F1** penalizes the 5 extra words and correctly prefers A.

Implementing ROUGE in Python

A From scratch: `rouge_from_scratch.py`

```python
from collections import Counter

def prf(o, ref_n, cand_n):
    R, P = o/ref_n, o/cand_n
    return P, R, (2*P*R/(P+R) if P+R else 0)

def rouge_n(ref, cand, n=1):
    g = lambda t: Counter(tuple(t[i:i+n])
                  for i in range(len(t)-n+1))
    r, c = g(ref.split()), g(cand.split())
    o = sum((r & c).values())       # clipped
    return prf(o, sum(r.values()), sum(c.values()))

def rouge_l(ref, cand):
    x, y = ref.split(), cand.split()
    dp = [[0]*(len(y)+1) for _ in range(len(x)+1)]
    for i in range(1, len(x)+1):
        for j in range(1, len(y)+1):
            dp[i][j] = dp[i-1][j-1]+1 if x[i-1]==y[j-1] \
                  else max(dp[i-1][j], dp[i][j-1])
    return prf(dp[-1][-1], len(x), len(y))
```

B Library: `pip install rouge-score`

```python
from rouge_score import rouge_scorer

scorer = rouge_scorer.RougeScorer(
    ["rouge1", "rouge2", "rougeL"],
    use_stemmer=True)

s = scorer.score("the cat sat on the mat",   # ref
                 "the cat lay on the mat")   # cand
for k, v in s.items():
    print(k, round(v.fmeasure, 3))
```

OUTPUT (BOTH A & B)

```
rouge1 0.833
rouge2 0.6
rougeL 0.833
```

Notes

- Arg order: **reference first**
- Use `rougeLsum` with sentences split by "\n"
- Stemming changes scores, so report it

05 · ADVANTAGES & LIMITATIONS

What ROUGE does well, and where it fails

Advantages

- **Fast & cheap**: no GPU, no human raters
- **Simple & transparent**: easy to explain and reproduce
- **Standard benchmark**: CNN/DailyMail, XSum, etc.
- **Coverage check**: recall shows whether key content is kept
- **Multi-reference**: can score against several human summaries

Limitations

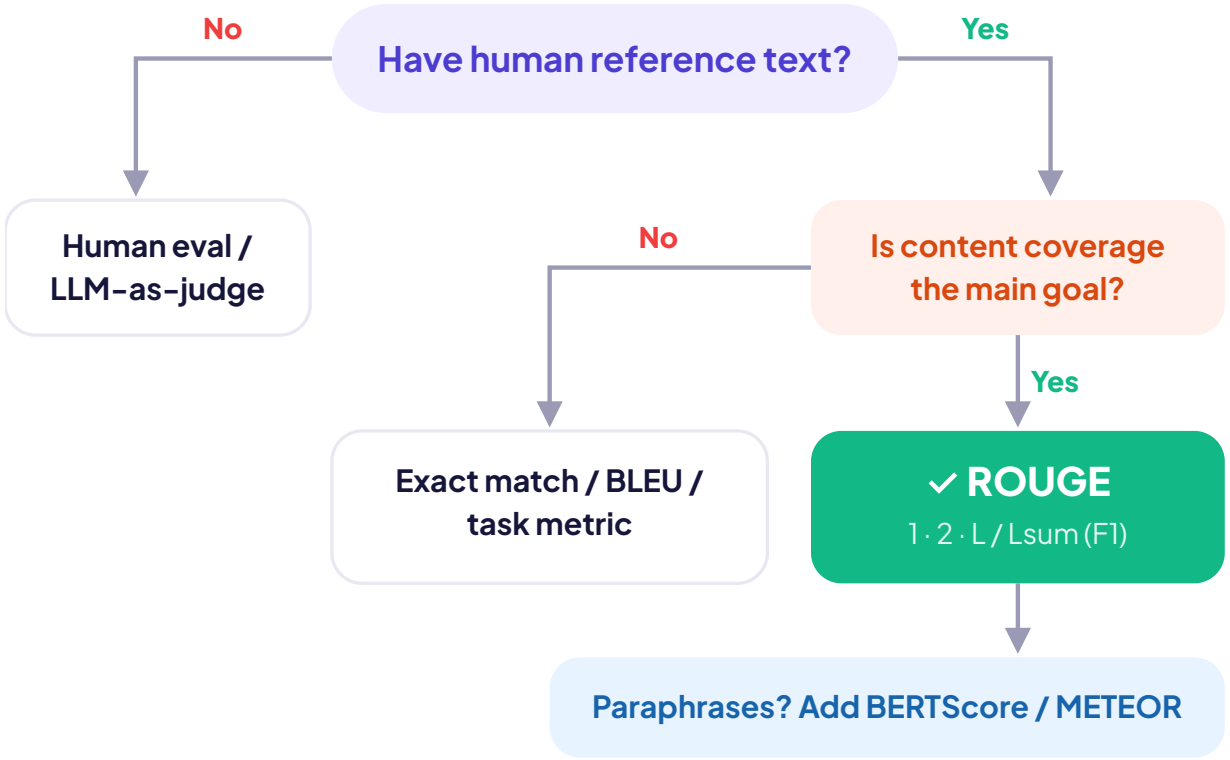
- **No semantics**: “car” ≠ “automobile”
- **Needs references**, and valid wording that differs is penalized
- **No fluency check**: grammar and coherence are ignored
- **Language-dependent**: Khmer has no spaces, so it needs word segmentation
- **Misses factual errors**: a negation can still score high
- **Recall favors long outputs** (Example 2)
- **Config-sensitive**: tokenizer and stemming change scores
- **Order-blind (ROUGE-1)**: see the case below

“the cat sat on the mat” → “the mat sat on the cat” ROUGE-1 = 1.000 ROUGE-2 = 0.800 ROUGE-L = 0.667

06 · CONCLUSION

When, why, and where to use ROUGE

WHEN · DECISION FLOW



WHY

A fast, reproducible check of **how much reference content** a model keeps. It's ideal for comparing models or catching regressions during development.

WHERE · APPLICATIONS

- News summarization
- Meeting & lecture notes
- Headline generation
- Report / paper abstracts
- Dialogue summarization
- Long-form QA answers
- LLM regression testing

Best practice: report ROUGE-1, ROUGE-2 and ROUGE-L F1 together, state your tokenizer and stemming settings, and pair ROUGE with a semantic or human check.

KEY TAKEAWAYS

01

ROUGE measures **n-gram and subsequence overlap** between LLM output and a human reference.

02

Report **ROUGE-1, ROUGE-2 and ROUGE-L F1** together. Each one catches different errors.

03

It's fast and standard but **blind to meaning**, so combine it with semantic or human evaluation.

Thank you

Questions & discussion

HANG Muykhorng · CHOU Kosol · NY Chantharith

Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. ACL Workshop.
Google Research `rouge-score` Python package